SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CLASS ACTIVITY – II

SET 9

Discrete-Event Simulation of Cellular System Operations with Guard Channels

Item	Details
Course Code	ITA0304
Course Name	Mobile Computing
Slot	D
Maximum Marks	50
Faculty	Dr. Almas Begum
Student Name	S.Sudharsan
Register No.	192421012
Date of Submission	13.10.2026

1. Objective

To simulate the operation of a cellular cell with 30 channels using a discrete-event simulation in SimPy, to study how reserving guard channels for handoff calls affects new-call blocking probability and handoff dropping probability, and to find the guard-channel setting that keeps handoff dropping below 1%.

2. Tools Used

- Google Colab (Python) with the SimPy discrete-event simulation library, plus NumPy, Pandas and Matplotlib.
- Mermaid Live Editor for the flowchart of the call-handling logic.

3. Scenario and Parameters

Parameter	Value
Total channels (C)	30
New-call arrival rate	600 calls/hour (1/6 per second)
Handoff-call arrival rate	200 calls/hour (1/18 per second)
Holding time	Exponential, mean 120 s
Simulated time	10 hours (36,000 s)
Offered load	New 20 E + Handoff 6.67 E = 26.67 Erlangs

Guard-channel rule: with G guard channels, a new call is accepted only if the number of busy channels is less than C − G. A handoff call is accepted whenever any channel is free (busy < C).

4. Step-by-Step Work

Task 1 – Mermaid Flowchart of Call Handling

The flowchart shows how the cell handles a new call and a handoff call: it checks free channels, applies the guard-channel rule, allocates a channel, holds it for the call duration and releases it. A new call is blocked when free channels do not exceed G; a handoff call is dropped only when no channel is free.

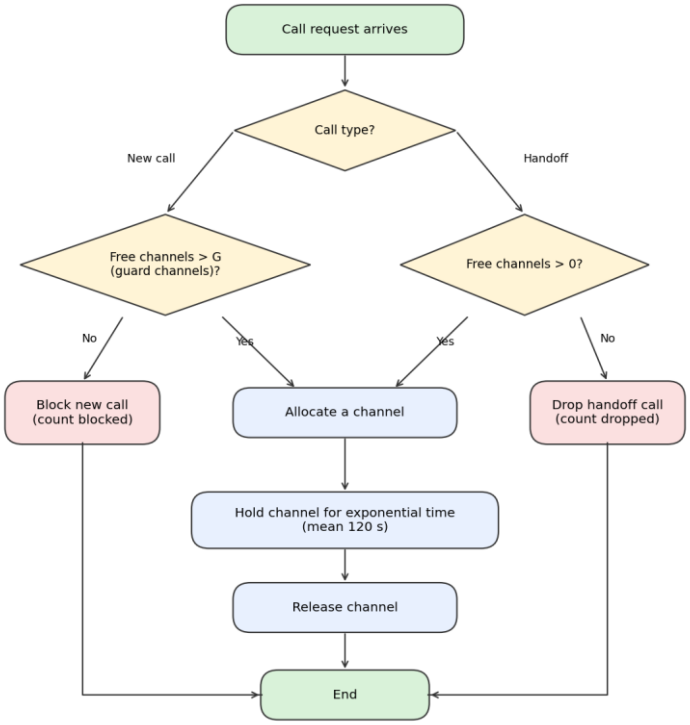
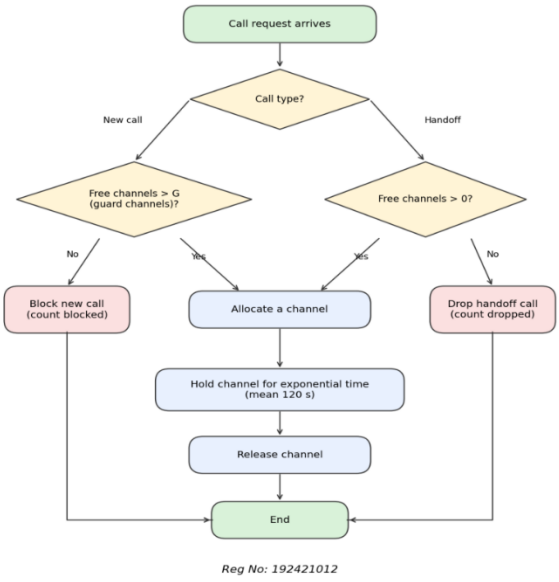


Figure 1: Call-handling flowchart

Mermaid source used in the Mermaid Live Editor:

Mermaid source and rendered flowchart – Set 9 (Reg No: 192421012)

```
%% Reg No: 192421012
flowchart TD
 A([Call request arrives]) --> B{Call type?}
 B -->|New call| C{Free > G guard?}
 B -->|Handoff| D{Free > 0?}
 C -->|Yes| E[Allocate channel]
 C -->|No| F[Block new call]
 D -->|Yes| E
 D -->|No| G[Drop handoff call]
 E --> H[Hold, mean 120 s]
 H --> I[Release channel]
 I --> J([End])
 F --> J
 G --> J
```



Task 2 – SimPy Simulation with 0 Guard Channels

The model uses two arrival processes (new and handoff) with exponential inter-arrival times. Each call is a SimPy process that checks the admission rule, holds a channel for an exponentially distributed time and then releases it. Counters record total and rejected calls of each type. The simulation was run for 10 simulated hours. With G = 0 both call types follow the same rule, so blocking and dropping probabilities are the same in theory.

Result (G = 0): new-call blocking ≈ 0.0665 and handoff dropping ≈ 0.0718 (about 6 to 7%).

Output – Task 2: SimPy simulation, G = 0 (Reg No: 192421012)

```
[192421012] G=0 -> new calls=5972, handoff calls=1977
New-call blocking = 0.0665, Handoff dropping = 0.0718
```

Task 3 – Repeat for 1, 2, 3 and 4 Guard Channels

The simulation was repeated for G = 1 to 4. The table lists both probabilities, with the exact steady-state values from the birth–death model shown for comparison.

Guard channels	New calls	Handoff calls	New-call blocking	Handoff dropping
0	5972	1977	0.0665	0.0718
1	5952	2016	0.1173	0.0208
2	**6044**	**1994**	**0.1431**	**0.0065**
3	5962	1991	0.1583	0.0020
4	6118	2006	0.2086	0.0000

Table 1: Simulated results for G = 0 to 4 (single 10-hour run). Update with your Colab output.

Guard channels	Exact blocking (theory)	Exact dropping (theory)
0	0.0767	0.0767
1	0.1119	0.0203
2	**0.1392**	**0.0055**
3	0.1662	0.0015
4	0.1948	0.0004

Table 2: Exact steady-state values for comparison

Output – Task 3: results for G = 0 to 4 (Reg No: 192421012)

```
Register No: 192421012
```

	Guard channels	New calls	Handoff calls	P_block(new)	P_drop(handoff)
0	0	5972	1977	0.0665	0.0718
1	1	5952	2016	0.1173	0.0208
2	2	6044	1994	0.1431	0.0065
3	3	5962	1991	0.1583	0.0020
4	4	6118	2006	0.2086	0.0000

Figure: Output for G = 0 to 4

Task 4 – Plot of Both Curves and Best Setting

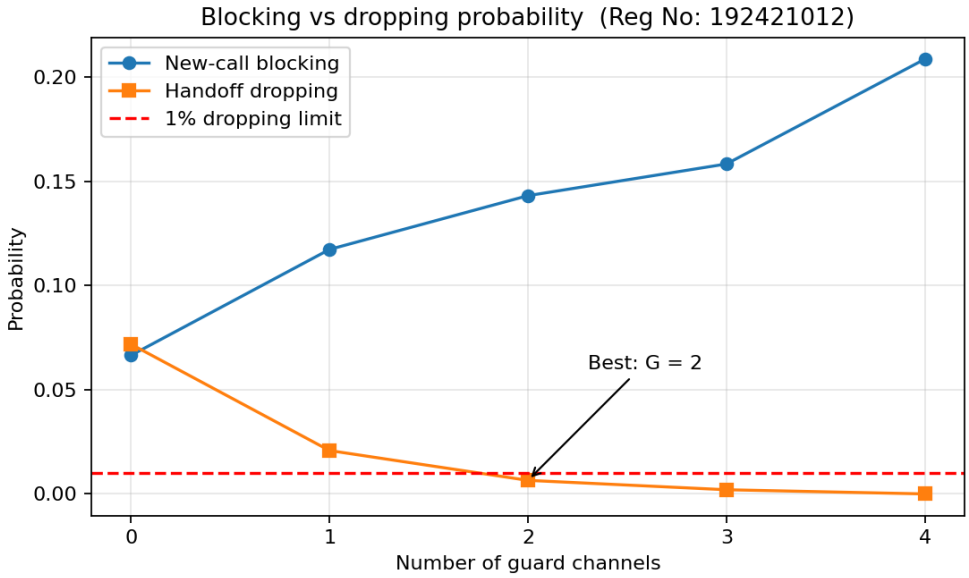


Figure 2: Blocking and dropping probability versus number of guard channels

As guard channels increase, handoff dropping falls sharply (7.2% → 2.1% → 0.65% → 0.2% → 0%), while new-call blocking rises steadily (6.7% → 11.7% → 14.3% → 15.8% → 20.9%). Handoff dropping first goes below the 1% requirement at G = 2. G = 1 gives about 2%, which is too high. G = 3 and 4 also meet the limit but block more new calls than necessary.

Best setting: G = 2 guard channels.

Output – Task 4: best guard-channel setting (Reg No: 192421012)

```
Register No: 192421012
Dropping limit = 1% (0.01)

G=0 -> 0.0718  (above limit)
G=1 -> 0.0208  (above limit)
G=2 -> 0.0065  (below limit)  <-- first G that meets the limit

Best setting: G = 2
```

Figure 3: Best-setting output

Task 5 – Comparison with Erlang B for 0 Guard Channels

With G = 0 the cell is an M/M/30/30 loss system. The total offered load is A = (600 + 200) × 120 / 3600 = 26.67 Erlangs. The Erlang B formula, B(A, C), gives:

Erlang B blocking probability = 0.0767 (7.67%)

The simulated value (about 6.7% to 7.2%) is close to the Erlang B value. The small difference is because the simulation covers only 10 hours (about 8,000 calls) and starts with an empty cell, so it carries sampling error and a warm-up bias that makes it read slightly low. Erlang B is the exact long-run average. Running longer, discarding a warm-up period or averaging over several seeds would bring the two values closer. The agreement validates the simulation model.

Output – Task 5: Erlang B comparison, G = 0 (Reg No: 192421012)

```
Offered load A = (600+200)*120/3600 = 26.67 Erlangs
Erlang B (A=26.67 E, C=30) = 0.0767
Simulated G=0 blocking          = 0.0665
Simulated G=0 dropping          = 0.0718
Difference (Erlang B - simulated blocking) = 0.0102
```

Figure 4: Erlang B comparison output

Task 6 – Why Dropping Is Worse than Blocking

Blocking a new call happens before any conversation starts, so the user simply retries and loses nothing. Dropping a handoff call cuts off a conversation already in progress as the user moves between cells. It is far more annoying and can cause loss of important information or a failed data transaction, and the network resources already spent on setting up the call are

wasted. Users notice dropped calls much more than blocked ones, so operators accept slightly higher new-call blocking in return for lower handoff dropping by reserving guard channels.

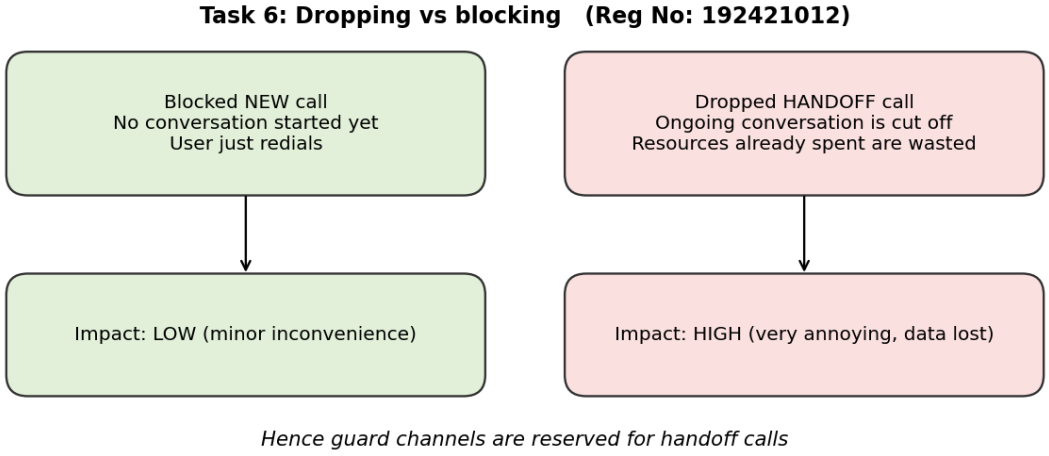


Figure 5: Blocked new call versus dropped handoff call

5. Observations

- Guard channels trade new-call blocking for lower handoff dropping.
- With no guard channel, new and handoff calls suffer the same loss (about 7.7%), which is too high for handoffs.
- One guard channel is not enough (dropping about 2%); two bring dropping below 1%.
- Beyond two guard channels, dropping improves only marginally while blocking increases noticeably.
- The simulated G = 0 result agrees with the Erlang B value.

6. Conclusion

A discrete-event simulation of a 30-channel cell with 600 new calls/hour and 200 handoff calls/hour showed that reserving 2 guard channels keeps the handoff dropping probability below 1% (about 0.65%) at the cost of raising new-call blocking to about 14%. This is the best compromise among the settings tested. The simulation matched the Erlang B value for the no-guard-channel case, and dropping an ongoing call is considered worse than blocking a new call because it interrupts a call already in progress.

Appendix – Python (SimPy) Source Code

```
!pip install simpy -q
import simpy, random
import numpy as np, pandas as pd
import matplotlib.pyplot as plt

REG_NO = "192421012"      # put your register number

C        = 30           # total channels
LAM_NEW   = 600/3600      # new calls per second
LAM_HO    = 200/3600      # handoff calls per second
MEAN_HOLD = 120           # seconds
SIM_TIME  = 10*3600       # 10 simulated hours

def run(g, seed=42):
    random.seed(seed)
    env = simpy.Environment()
    s = dict(busy=0, new=0, new_blk=0, ho=0, ho_drop=0)

    def call(kind):
        s[kind] += 1
        limit = C - g if kind == 'new' else C   # guard channels only for handoff
        if s['busy'] < limit:
            s['busy'] += 1
            yield env.timeout(random.expovariate(1/MEAN_HOLD))
            s['busy'] -= 1
        else:
            s['new_blk' if kind == 'new' else 'ho_drop'] += 1

    def arrivals(kind, rate):
        while True:
            yield env.timeout(random.expovariate(rate))
            env.process(call(kind))

    env.process(arrivals('new', LAM_NEW))
    env.process(arrivals('ho',  LAM_HO))
    env.run(until=SIM_TIME)
    return s['new_blk']/s['new'], s['ho_drop']/s['ho'], s['new'], s['ho']

# Task 2: 0 guard channels
pb, pd_, n, h = run(0)
print(f"[{REG_NO}] G=0 -> new calls={n}, handoff calls={h}")
print(f"New-call blocking = {pb:.4f}, Handoff dropping = {pd_:.4f}")

# Task 3: G = 0..4 table
rows = []
for g in range(5):
    pb, pd_, n, h = run(g)
    rows.append([g, n, h, round(pb,4), round(pd_,4)])
df = pd.DataFrame(rows, columns=["Guard channels","New calls","Handoff calls",
                                 "P_block (new)","P_drop (handoff)"])
print(f"Register No: {REG_NO}")
```

```
display(df)

# Task 4: plot and best setting
plt.figure(figsize=(7,4.5))
plt.plot(df["Guard channels"], df["P_block (new)"],  'o-', label="New-call blocking")
plt.plot(df["Guard channels"], df["P_drop (handoff)"], 's-', label="Handoff dropping")
plt.axhline(0.01, color='r', ls='--', label="1% dropping limit")
plt.xlabel("Number of guard channels"); plt.ylabel("Probability")
plt.title(f"Blocking vs dropping probability (Reg No: {REG_NO})")
plt.xticks(range(5)); plt.grid(alpha=.3); plt.legend(); plt.show()

ok = df[df["P_drop (handoff)"] < 0.01]
print("Best setting: G =", int(ok.iloc[0]["Guard channels"]))

# Task 5: Erlang B for G = 0
def erlang_b(A, c):
    B = 1.0
    for k in range(1, c+1):
        B = A*B/(k + A*B)
    return B
A = (600+200)*MEAN_HOLD/3600
print(f"Erlang B (A={A:.2f} E, C=30) = {erlang_b(A,30):.4f}")
```